COMPASSION PORTAL SYSTEM STRUCTURE

Analysis date: 2026-04-14

1. System overview

Compassion Portal is a Laravel 13 web application used to manage participants, sponsorship records, users, centers, and center-based notifications. The system follows a multi-center access model where users only see data for the centers they are allowed to access. It uses Blade templates for the UI, SQLite in the current local environment, and Vite plus Tailwind CSS for frontend assets.

2. Core technology stack

- Backend framework: Laravel 13
- Backend language: PHP 8.3+
- Frontend rendering: Blade templates
- Frontend tooling: Vite
- Styling: Tailwind CSS
- JavaScript helpers: Alpine.js, Axios
- Authentication foundation: Laravel Breeze
- Local database: ‘database/database.sqlite‘

3. Important folder structure

```
compassion-portal/
|-- app/
| |-- Http/
| | |-- Controllers/
| | | |-- Admin/
| | | ‘-- Auth/
| | |-- Middleware/
| | ‘-- Requests/
| |-- Models/
| |-- Providers/
| |-- Services/
| ‘-- View/Components/
|-- bootstrap/
|-- config/
|-- database/
| |-- migrations/
| |-- factories/
| ‘-- seeders/
```

```
|-- public/
|-- resources/
| |-- css/
| |-- js/
| '-- views/
| |-- admin/
| |-- auth/
| |-- components/
| |-- dashboard/
| |-- layouts/
| |-- notifications/
| |-- participants/
| |-- profile/
| '-- sponsorships/
|-- routes/
| |-- web.php
| '-- auth.php
|-- scripts/
|-- storage/
'-- tests/
```

4. Main business modules

4.1 Authentication and security

- Login, registration, forgot password, and reset password are defined in 'routes/auth.php'.
- After login, the system enforces a second step of OTP verification through email.
- The 'otp' middleware blocks access to the dashboard and protected modules until OTP is verified.
- Role-based middleware in the project includes:
- 'EnsureAdmin'
- 'EnsureOfficialAdmin'
- 'EnsureOtpVerified'

4.2 Dashboard

- 'DashboardController' collects participant, sponsorship, user, and notification statistics.
- The dashboard changes based on role:
- Official Admin: sees all centers
- Admin: sees assigned managed centers
- User: sees only the user center

4.3 Participant management

- ‘ParticipantController‘ handles:
- participant listing
- quick search
- participant creation
- participant editing and updating
- participant deletion
- single participant view
- A participant stores profile, education, spiritual, medical, address, and favorites information.
- The system generates ‘local_participant_id‘ from ‘center_id‘ plus a running serial number.
- Photo upload stores the image in public storage and sets the next photo update due date to six months later.

4.4 Sponsorship management

- ‘ParticipantSponsorshipController‘ is a standalone module for sponsorship records.
- Sponsorship records are linked to participants through ‘participant_id‘.
- When sponsorship data is created or updated, the participant summary fields can also be synchronized.

4.5 Center notifications

- ‘CenterNotificationService‘ creates center notifications based on:
- overdue photo updates
- photo updates due within 30 days
- recent participant updates
- newly created participants
- ‘CenterNotificationController‘ shows notifications and supports marking one or all as read.

4.6 Admin management

- ‘AdminDashboardController‘ provides administrative metrics.
- ‘UserManagementController‘ handles:
- user creation
- user editing
- password reset
- center assignment
- ‘OfficialAdminController‘ provides a system-wide overview across all centers.
- ‘CenterReportController‘ provides reports and CSV exports for:
- participants
- users
- notifications

5. Core models and relationships

5.1 User

Responsibilities:
- authentication
- role management
- center access control
- OTP ownership
- notification read tracking

Defined roles:
- ‘official_admin‘
- ‘admin‘
- ‘user‘

Relationships:
- a user belongs to one center through ‘center_id‘
- a user has many OTP codes
- an admin user can manage many centers through ‘center_user_assignments‘
- a user has many notification reads

5.2 Center

Responsibilities:
- storing the center identity
- linking users and assigned admins to a center

Relationships:
- a center has many users
- a center belongs to many assigned admin users

5.3 Participant

Responsibilities:
- storing complete participant information
- tracking participation status
- tracking photo lifecycle
- linking to sponsorship records

Relationships:
- a participant belongs to one center
- a participant has many sponsorships
- a participant has one latest sponsorship

5.4 ParticipantSponsorship

Responsibilities:

- storing funding type
- sponsorship status
- sponsor details
- sponsorship start date
- sponsorship category

Relationships:
- a sponsorship belongs to a participant

5.5 CenterNotification

Responsibilities:
- storing center-level notifications
- storing due dates and notification metadata

Relationships:
- a notification belongs to a participant
- a notification has many reads

5.6 OtpCode

Responsibilities:
- storing the verification code
- storing expiry time
- tracking used or unused state

6. High-level database structure

Main tables visible from the migrations:

- ‘users‘
- ‘otp_codes‘
- ‘centers‘
- ‘participants‘
- ‘participant_sponsorships‘
- ‘center_notifications‘
- ‘center_notification_reads‘
- ‘center_user_assignments‘
- ‘cache‘
- ‘jobs‘

High-level data relationships:

centers
|--< users
|--< participants

users
|--< otp_codes
|--< center_notification_reads
‘--< center_user_assignments >-- centers

participants
|--< participant_sponsorships
‘--< center_notifications

center_notifications
‘--< center_notification_reads

7. System request flow

7.1 Login mpaka dashboard

1. The user logs in through the login form.
2. If credentials are valid, the system redirects the user to OTP verification.
3. The OTP is sent by email and stored in ‘otp_codes‘.
4. When the user verifies the OTP, the session gets ‘otp_verified = true‘.
5. The user can then access the dashboard and other routes protected by ‘auth‘ and ‘otp‘.

7.2 Participant create/update flow

1. The user opens the participant form.
2. The system validates the submitted fields.
3. The center scope is determined from the authenticated user.
4. A local participant ID is generated.
5. If a photo is uploaded, the file is stored and the next photo due date is set.
6. Sponsorship summary data may be synchronized to the participant.
7. A create or update notification may be generated.

7.3 Access control flow

1. The system checks the user role.
2. The system resolves ‘accessibleCenterIds()‘.
3. Most queries are filtered through the ‘forCenter(...)‘ scope.
4. This prevents cross-center data access.

8. Views and UI structure

The views are organized by module:

- ‘resources/views/layouts‘: main layouts, guest layout, navigation
- ‘resources/views/auth‘: login, register, OTP verification, password flows
- ‘resources/views/dashboard‘: main dashboard
- ‘resources/views/participants‘: list, create, edit, show
- ‘resources/views/sponsorships‘: list, create, edit
- ‘resources/views/notifications‘: notification center
- ‘resources/views/admin‘: admin dashboard, users, reports, official overview
- ‘resources/views/components‘: reusable Blade components

9. Services and reusable logic

The main service class currently visible is:

- ‘CenterNotificationService‘

This service moves business logic out of controllers, especially for notification synchronization related to due dates and recent participant updates.

10. Architectural strengths

- It follows standard Laravel conventions, which makes the codebase easier to maintain.
- There is a clear separation between routes, controllers, models, views, middleware, and services.
- Center-level access control is applied through query scopes and helper methods.
- OTP adds extra protection beyond normal login.
- Notification generation has already been extracted into a dedicated service class.

11. Improvement opportunities

- Move long participant validations into dedicated Form Request classes.
- Add tests for authorization, OTP flow, and notifications.
- Introduce more service classes if the business logic continues to grow.
- Add reporting abstractions if exports and analytics become more complex.
- Consider queues for OTP emails and notification-heavy work in production.

12. Conclusion

Compassion Portal is a Laravel monolith organized for center-based participant operations. Its current structure supports role-based access, center-scoped visibility, participant record management, sponsorship tracking, admin oversight, and notification handling for time-sensitive events such as photo due dates.